

Analysis of Universal Circuits in a CSP Transformer

Apr. 14, 2026

Krishnan, Asha¹, Mohanarajan, Vignesh¹, Sahin, Efe¹, Wilam, Piotr¹, Yeung, Matt¹

¹University College London, London, United Kingdom

{asha.krishnan.24, vignesh.mohanarajan.25, efe.sahin.25,
piotr.wilam.25, matthew.yeung.23}@ucl.ac.uk

Abstract

Although Large Language Models are remarkably capable, they are highly uninterpretable, meaning that their internal feature representations are highly entangled. Prior work has particularly studied mechanistic interpretability of dense models under unconstrained pre-training settings. In our study, we apply this lens to a sparsely activating code language model (401M parameters) trained by OpenAI, constructing a synthetic contrastive dataset of 14,683 prompts spanning the principal Python concepts (AST node, built-in) pairs across six surface variations, and running two complementary experiments: variance partitioning and sparse autoencoders. Both methods independently surface a small number of strictly specialised units, including a single MLP neuron exclusively encoding `Assert` and an attention head tuned almost entirely to `import` statements, alongside residual-stream dimensions that cluster constructs by conceptual role rather than surface syntax. Our findings suggest that sparse activation creates partial disentanglement of distinct Python concepts but superposition remains the dominant encoding strategy.

1 Introduction

Large Language Models (LLMs) have proven remarkably effective across complex reasoning tasks. Yet whether these models meaningfully represent structured knowledge, or merely exploit statistical patterns, remains poorly understood. Mechanistic interpretability is the study of how LLMs internally store and process learned features, and the scale and complexity of modern LLMs makes this a difficult undertaking.

Superposition hypothesis (Elhage et al., 2022) explains why individual neurons are insufficient to represent distinct learned features: rather than each neuron encoding one concept, features are

distributed across many neurons simultaneously. This phenomenon arises because the number of learnable features in a training corpus far exceeds the number of neurons a model can accommodate, forcing the network to compress multiple features into shared dimensions. As a result, circuits become entangled and difficult to isolate, as multiple components share responsibility for any given behaviour.

Interpretability has been applied across a range of model types, from general-purpose text models (Elhage et al., 2021; Olsson et al., 2022) to probing studies of cross-lingual and cross-domain representations (Tenney et al., 2019; Belinkov, 2022). Recently, code LLMs have gained traction as targets for interpretability research, with sparse activation constraints emerging as a specific promising design choice. OpenAI’s weight-sparse transformer (Bricken et al., 2024) reduces the degree of superposition per input, making it easier to isolate circuits responsible for core coding constructs such as loops, conditionals and function definitions. This sparsity is the central experimental affordance of our study and if dedicated circuits exist anywhere, a sparsely activating model is where they are most likely to surface.

In our study we investigated the principal concepts naturally present in Python, distinguishing between two fundamental categories: Abstract Syntax Tree (AST) nodes and built-in objects as these are two distinct concepts Python relies on to parse, interpret and execute a given program. We then asked the question of whether AST-syntactic and built-in-semantic features separate, overlap or interact throughout the representation space of this model. To answer this, we employed variance partitioning (Nimon et al., 2008) and sparse autoencoders (SAEs) (Cunningham et al., 2023) as complementary lenses. We find that sparsity creates pockets of genuine disentanglement, including a monosemantic neuron, a head tuned almost

exclusively to import statements, and residual dimensions organised by conceptual role, yet superposition remains the dominant encoding strategy throughout the model.

2 Background

Large Language Models trained on code have proven remarkably capable at synthesis, completion, and reasoning over programs (Chen et al., 2021; Rozière et al., 2023), yet how these models internally encode structural and semantic program properties remains poorly understood. A Python snippet is simultaneously characterised by its syntactic structure: AST node types such as `For`, `If`, or `FunctionDef`, and its semantic objects, such as built-in functions (`len`, `sorted`, `range`). Whether these dimensions are encoded separately, in overlapping units, or through interaction circuits has direct implications for both interpretability and model editing.

Mechanistic Interpretability analyses transformer architectures at the level of individual components. Elhage et al. (2021) formalised the residual stream as an additive communication channel, making it a natural focus for representation learning. Elhage et al. (2022) showed that networks store more features than dimensions by encoding them as sparse, near-orthogonal directions, therefore causing individual neurons to respond to multiple unrelated concepts (polysemanticity). Circuit-level work has identified motifs such as induction heads (Olsson et al., 2022), and ACDC (Conmy et al., 2023) which extends this by isolating the minimal subgraph sufficient for a given behaviour.

The model studied here is OpenAI’s `circuit_sparsity` transformer (CSP) (Gao et al., 2025), a 401M-parameter Python code model using AbsTopK activation, where only the k neurons with the largest absolute value remain active per forward pass. Figure 1 previews the main structural finding: of 1,024 attention heads, only 53 are strictly monosemantic which is a direct manifestation of superposition despite the model’s sparsity constraint.

Probing classifiers offer a complementary view: a linear classifier on frozen hidden states measures what information is linearly accessible at each layer (Belinkov, 2022). Tenney et al. (2019) showed syntactic structure emerges in early layers while semantic roles appear later. For code

models, Feng et al. (2020) and Guo et al. (2021) demonstrated sensitivity to data-flow and type structure. Both treat syntactic and semantic signals jointly, which raises the question on whether the model separates or conflates them.

We extend probing to a factorial design, training classifiers separately on AST-pure, builtin-pure, and shared subspaces to test for modular encoding versus cross-factor leakage. To quantify separation across every model component, we employ variance partitioning (VP), which decomposes each component’s R^2 into four additive terms, namely unique to AST, built-in, shared, and unexplained (Nimon et al., 2008; Kriegeskorte et al., 2008). This yields a purity map over all residual-stream dimensions, MLP neurons, and attention heads.

Since VP is correlational, causal verification typically requires activation patching (Meng et al., 2022; Heimersheim and Nanda, 2024), which replaces a component’s activations with those from a counterfactual prompt to confirm that the component is necessary for the observed behaviour. We do not perform patching here; instead, a complementary causal signal is obtained via Sparse Autoencoders (SAEs), which recover nearly monosemantic features by training an overcomplete dictionary under an L_1 penalty (Cunningham et al., 2023). This approach has scaled to production models (Templeton et al., 2024; Gao et al., 2024), with gated variants improving separation (Rajamanoharan et al., 2024), and SAE features can be composed into circuits that are both interpretable and causally verified (Marks et al., 2024). Variance partitioning and sparse autoencoders together form the two complementary analyses developed in Section 3.

3 Methodology

Our analysis proceeds in three stages: VP (Section 3.1), stimulus generation (subsubsection 3.1.1), and sparse autoencoder feature discovery with causal validation (Section 3.3). Throughout, we combine representational similarity analysis with minimum-spanning-tree visualisation of class-level cosine-similarity graphs to characterise the geometry of syntactic representations across depth.

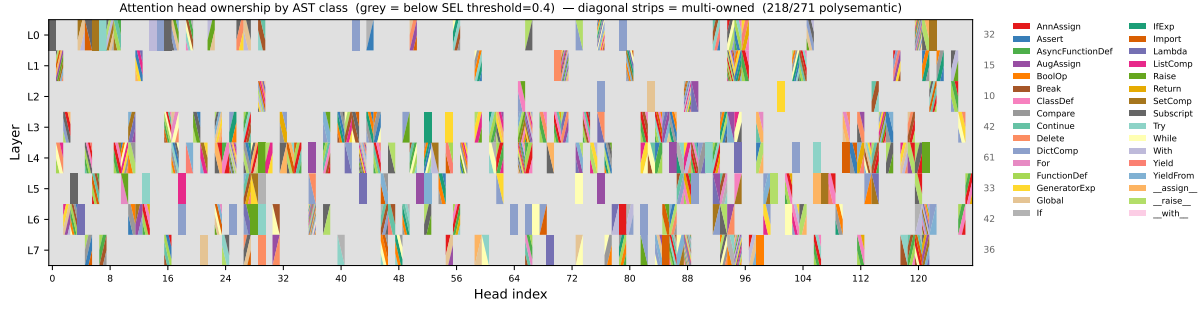


Figure 1: Attention head ownership across all layers. Solid cells: monosemantic heads. Split-colour cells: heads shared by multiple AST classes.

3.1 Variance Partition Methodology

In this paper, for factors A (AST node) and B (built-in), given prompt u , define:

$$\text{unique}_A(u) = v^{\text{ast}} = R_{AB}^2(u) - R_B^2(u), \quad (1)$$

$$\text{unique}_B(u) = v^{\text{b}} = R_{AB}^2(u) - R_A^2(u), \quad (2)$$

$$\text{shared}(u) = R_A^2(u) + R_B^2(u) - R_{AB}^2(u), \quad (3)$$

where R_{AB}^2 is the joint explained variance. This yields a four-way decomposition over every residual-stream dimension and MLP neuron, identifying *pure AST* units ($\text{unique}_A \gg \text{shared}$), pure builtin units, shared/interaction units, and noise. See Appendix B for details.

Contrastive design and permutation significance. Variance is partitioned *relative* to the contrast across $C=32$ AST classes rather than in absolute terms, shifting the analysis to *differential* interpretability: VP isolates which units drive the difference between classes, cancelling shared background variance (syntax, positional encoding, token statistics). For comprehension and subscript nodes, the design is not strictly contrastive in surface form as all are rewritten to constructor form ending in ‘)’ to expose deeper structural signals rather than delimiter artefacts (see subsubsection 3.1.1). Permutation tests (5,000 class-label relabellings) provide a clean empirical null; surviving units encode structure beyond chance in a balanced $C=32$ design.

3.1.1 Constructor-Stubs Dataset

We construct 14,683 short Python snippets spanning 32 AST node classes. Prompts are designed to minimise three confounds: *syntactic bias* (six lexical variants per pair suppress identifier-level noise), *token-prediction bias* (comprehension and subscript nodes are rewritten to constructor form

so every prompt ends with ‘)’), avoiding shared closing-delimiter signals at the final extraction token), and *built-in co-occurrence* (constructs that unavoidably require a built-in are paired across 1,405 (AST, built-in) combinations with baseline and proxy controls; constructs that can be expressed without a built-in are written without one). Full dataset structure, bias-avoidance rationale, and lexical variant details are given in Section A.2.

3.2 Activation Grouping

AST classes are clustered hierarchically using Ward linkage on the correlation distance between their mean selectivity profiles across all active MLP and residual stream dimensions. Groups can be identified by cutting dendrogram into k groups. For each group $\mathcal{G}$ and each active unit (l, n) , we compute the *group selectivity score*

$$\Delta \bar{s}_{\mathcal{G}, l, n} = \frac{1}{|\mathcal{G}|} \sum_{c \in \mathcal{G}} s_{c, l, n} - \frac{1}{C - |\mathcal{G}|} \sum_{c \notin \mathcal{G}} s_{c, l, n} \quad (4)$$

and the within-group activation standard deviation $\sigma_{\mathcal{G}, l, n} = \text{std}_{c \in \mathcal{G}}(s_{c, l, n})$. A unit is declared group-selective if $\Delta \bar{s} > 0.5$ (specificity), $\sigma_{\mathcal{G}} < 0.8$ (uniform activation across all group members as this criterion rules out units driven by a single dominant class within the group and is therefore the key distinction from the individual-class analysis), and $p < 0.01$ under a permutation test with 5000 random relabellings of classes into groups of the same size.

3.3 Sparse Auto-Encoders

The CSP transformer enforces sparsity at an architectural level, it is enforced over units not concepts - this means that its native activations remain polysemantic. We introduce *Sparse Autoen-*

coders (SAEs) as an interpretability tool to disentangle overlapping features encoded across the residual stream and MLP outputs. Separate SAEs are trained for each layer (0–7) and for two activation sites namely the MLP output before the residual addition and the post-block residual stream, thus yielding 16 SAEs in total. All SAEs are trained on cached activations using reconstruction MSE, with model selection based on held-out R^2 .

Feature discovery follows a factorial protocol on the 14,683-prompt constructor-stubs dataset, stratified 50/50 into discovery and held-out splits. For each (layer, site, AST node), the top-1 feature *winner* is selected on the discovery split by mean-activation gap against all other explicit prompts, and validated on the held-out split via four controls: (i) Mann–Whitney U and Cohen’s d against other constructs; (ii) `baseline_ast` and `baseline_builtin` prompts that separate AST identity from builtin co-occurrence; (iii) `proxy` rewrites that distinguish concept-level features from surface-token artefacts; and (iv) cross-dataset replication on the 8,897-prompt `contrastive_stubs` dataset. Winners are benchmarked against a null distribution of 500 random SAE features.

To test whether these winners encode the same structure revealed by a geometry-only view, an independent dendrogram is built from the cosine distances between mean per-construct residual activations, yielding four coarse clusters (control flow, predicates, definitions, mutations/ comprehensions/yields) and six tight pairs at distance ≤ 0.5 . SAE winners are then compared against this reference structure across all 16 (layer, site) combinations. Full architectural and discovery details are given in [Appendix C](#).

4 Results

4.1 Neuron, Head and Structure Identification using Variance Partitioning

VP surfaces three tiers of selectivity: a handful of strictly monosemantic MLP neurons, a cohort of exclusively-tuned attention heads, and hundreds of residual-stream dimensions selective for conceptually cohesive AST families at the group level. Superposition appears to be the rule with a few exceptions.

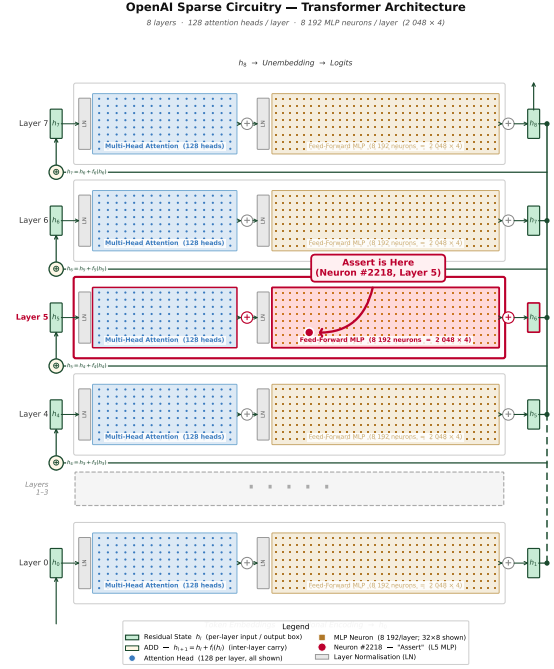


Figure 2: Monosemantic MLP neuron exclusively selective for the `Assert` class across all 32 AST classes (VP analysis)

4.1.1 Monosemantic MLP Neurons

Whilst the majority of MLP neurons were found to be polysemantic, three satisfied strict monosemanticity across all 32 AST classes (full table in [Section D.2](#)). Neuron 2218 at layer 5 is by far the most prominent: it fires 2.84σ above the mean for `Assert` with exactly zero activation on all 31 other classes, confirmed by VP (unique-AST = 0.241, $R^2_{\text{joint}} = 0.267$) and illustrated in [Figure 2](#). The two remaining neurons show substantially weaker selectivity ($\text{sel} < 1.3$) and are likely enabled by the same AbsTopK gating mechanism that produces the hard silencing seen in N2218. Full identification methodology is given in [Appendix D](#).

4.1.2 Attention Head Class Selectivity

Of the 1024 attention heads, 316 (31%) pass the AST-purity filter (see [Appendix B](#) for thresholds). Of these, 271 (86%) respond selectively to at least one AST class, and 218 (80% of owned heads) respond to two or more classes simultaneously which is a signature of superposition, and consistent with the finding that conceptually related ASTs share heads ([Figure 1](#), [Figure 6](#)). Only 53 heads are strictly monosemantic (single owner). To confirm that these 53 monosemantic heads are

not merely threshold artefacts, we inspect the selectivity gap $\Delta = \text{sel}_{\text{top1}} - \text{sel}_{\text{top2}}$: the median gap is 1.47, and 40/53 heads have $\Delta > 0.3$. The standout case is L4H110 (Import, $\text{sel} = 12.14$, runner-up Delete at 0.11, $\Delta = 12.03$), a head tuned so exclusively to import statements that it dwarfs every other finding by an order of magnitude. Full ranked list and selection details are given in [Section E.1](#).

4.1.3 Group-Selective Residual Stream Dimensions

The residual stream is essentially a shared supposition state accumulating contributions from every preceding layer. No strictly monosemantic per-class residual unit is found: any given dimension carries information for many concurrent computations, making single-class selectivity effectively impossible at this level. At the group level, however, clear structure emerges: dimensions that co-activate uniformly for semantically cohesive construct families are identified via a group-selectivity score and permutation test ($p < 0.01$, 5,000 re-labellings); full per-group results including permutation p -values are in [Table 6](#).

Ward-linkage clustering at $k=14$ yields ten groups with significant selective units; three singletons (BoolOp, Import, `__with__`) produce none. Two groupings are especially revealing.

{GeneratorExp, ListComp, SetComp} — iteration semantics (1,097 dims). All three share the same `expr for var in iterable` skeleton and produce flat, non-keyed output. In constructor form ([Listing 1](#)), `list(B for I in D)`, `set(B for I in D)`, and the bare generator are token-sequence near-twins, suggesting the model encodes a *comprehension* concept independent of output type.

{DictComp, Lambda} — transformation semantics (563 dims). Both encode a *functional transformation* — given an input, compute a derived output by rule — whereas `ListComp/SetComp` primarily *collect*. `dict((k, B) for k, I in enumerate(D))` and `lambda I: B` both say “map each input to a result”; `list(B for I in D)` says “gather results”. The model appears to have learnt this functional boundary, not the syntactic one.

```
1 data = [1, 2, 2, 3, 3, 3, 4]
2
3 # G2: iteration / collection
4 result = list(item for item in data)
5 result = set(item for item in data)
6 gen     = (item for item in data)
7
8 # G5: transformation / mapping
9 result = dict((k, item) \
10              for k, item in enumerate(data))
11 fn     = lambda item: item * 2
```

Listing 1: Constructor-form prompts (all end with `)`). G2 share an iteration skeleton; G5 share transformation semantics.

Constructor form exposes G2’s shared skeleton and severs `DictComp`’s `{}`-delimiter association with `SetComp`, revealing the G5 mapping cluster. The trade-off is reduced idiomaticity — `list(x for x in data)` is rarely written in practice — and the `enumerate` call in `DictComp` introduces an extra built-in co-occurrence. Whether groupings reflect genuine model semantics or are partially shaped by these prompt choices remains an open question.

4.2 SAE Winners Corroborate Dendrogram Structure Selectively

To test whether SAE-discovered features encode the same structure an independent geometric view surfaces, we (i) hierarchically cluster the 29 AST constructs by cosine distance over their mean residual activations, obtaining four coarse clusters ([Figure 4](#)): *control flow* (cyan), *predicates* (red), *definitions* (yellow), *mutations/comprehensions/yields* (purple) — and six *tight pairs* at dendrogram distance ≤ 0.5 ; and (ii) for each of the 16 (layer, site) combinations, assign every construct its top-1 SAE *winner* feature on a held-out split of the 14,683-prompt `constructor_stubs` dataset. A tight pair is *corroborated* at (layer, site) when both members share the same winner; a cluster is *bundled* when two or more members do. Across all 16 (layer, site) combinations and 29 constructs, 374/464 (80.6%) of the winners replicate on the held-out `contrastive_stubs` dataset with $p < 0.05$ and matching direction, and 412/464 (88.8%) reach the 95th percentile or above of the 500-feature random-null specificity distribution. Effect sizes grow with depth and peak at mid-late layers: median Cohen’s d rises from 0.99 at L0 residual to 2.88 at L6 residual before settling to 2.03 at L7 residual (max 10.65);

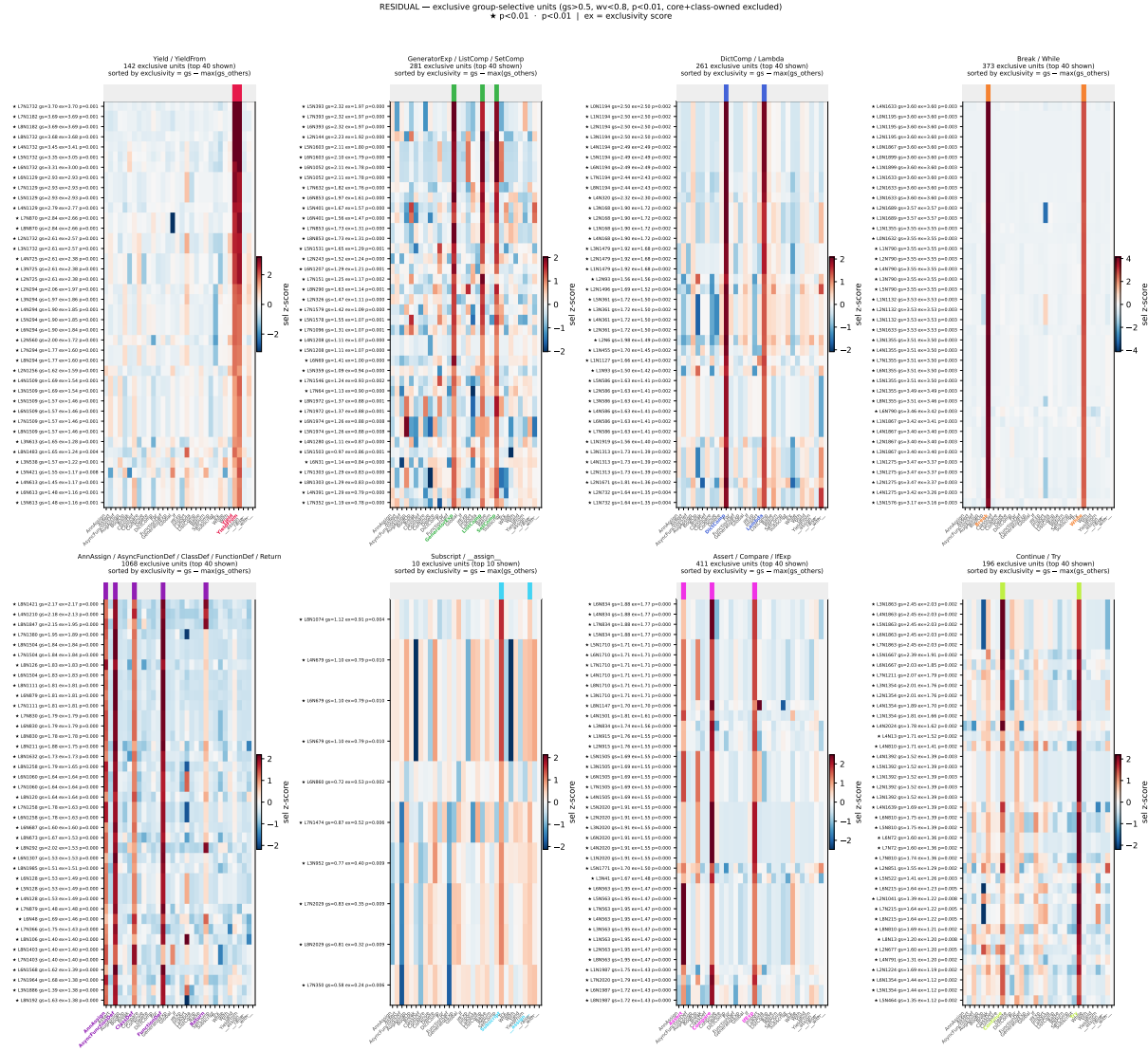


Figure 3: Residual stream dimensions exclusively co-activated by syntactically related AST groups. Each panel shows one group; rows are units sorted by exclusivity score.

the MLP site follows the same pattern, peaking at $d=1.99$ at L6 MLP.

Tight-pair corroboration tracks dendrogram tightness. The tightest pair $\text{FunctionDef} \leftrightarrow \text{AsyncFunctionDef}$ (cos.dist. 0.04) shares a winner feature at *every* layer and both sites (16/16). $\text{ListComp} \leftrightarrow \text{GeneratorExp}$ (0.07) is corroborated at 11/16, and $\text{AnnAssign} \leftrightarrow \text{ClassDef}$ (0.14) at 12/16. The weakest pairs are rarely bound: $\text{Compare} \leftrightarrow \text{IfExp}$ (0.36) at 4/16, and $\text{If} \leftrightarrow \text{For}$ (0.48) at 5/16 (Table 1). Corroboration therefore scales with geometric proximity rather than surface-token overlap, i.e the SAE independently recovers the nearest-neighbour structure revealed by the cosine-only view.

Table 1: Tight dendrogram pairs: number of layers (out of 8) at each SAE site where both constructs share the top-1 winner feature.

Pair	Cos.dist.	MLP	Resid
$\text{FunctionDef} \leftrightarrow \text{AsyncFunctionDef}$	0.04	8/8	8/8
$\text{ListComp} \leftrightarrow \text{GeneratorExp}$	0.07	5/8	6/8
$\text{AnnAssign} \leftrightarrow \text{ClassDef}$	0.14	8/8	4/8
$\text{While} \leftrightarrow \text{Break}$	0.20	6/8	4/8
$\text{Compare} \leftrightarrow \text{IfExp}$	0.36	3/8	1/8
$\text{If} \leftrightarrow \text{For}$	0.48	2/8	3/8

4.3 Polysemantic Bundling Peaks Early and Relaxes with Depth

Cluster-level bundling — the largest number of within-cluster constructs that share a single win-

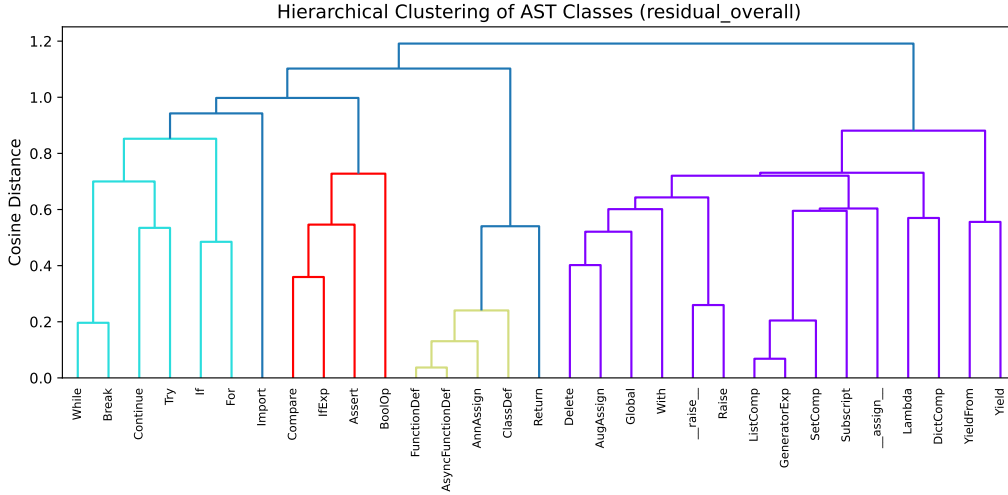


Figure 4: AST class grouping using Residual Stream neuron activations – aggregated across all layers.

ner feature — reveals an interesting phenomenon through the layers. The *purple* cluster (13 constructs) is the most polysemantic: at residual layer 2 all 13 members collapse onto a single winner feature, and at MLP layer 3 ten of thirteen do. Purple bundles then contract with depth, dropping to ≤ 3 at residual layers 5–7 and MLP layers 6–7, consistent with the model refining broad abstractions into narrower features as representations consolidate. *Yellow* (5 definition constructs) behaves differently: four of five members bind to one MLP feature at *every* layer from L1 to L7, and four of five under one residual feature at L0–L2 — definitions remain persistently entangled rather than refined apart. *Cyan* (7 control-flow constructs) peaks at MLP L1–L2 with bundle size 5 and relaxes to 2, while *red* (4 predicates) never exceeds bundle size 3 and collapses to zero at MLP layers 5–7, a cluster too heterogeneous for any single feature to own.

Factorial controls flag residual builtin contamination. Each winner is classified by the factorial protocol: if a feature fires on `baseline_ast` (construct retained, builtin stripped) and is silent on `baseline_builtin` (builtin retained, construct absent) it is *AST-specific*; the inverse is *Builtin-specific*; both-active is *Both-respond*; both-silent-but-responsive-to-explicit is *Joint-required*; proxy rewrite retention marks *Concept*, otherwise *Surface-token*. AST-specific winners dominate at the residual site: 15/29 at L4, 20/29 at L5 and L6, 15/29 at L7. MLP winners are more mixed — AST-specific counts range 10–17/29 across L4–L7 (peaking at L6), while

Both-respond counts reach 11/29 at L7. At every deep (layer, site) except L6 resid, 1–3 winners are flagged Builtin-specific (e.g. `Yield` at L7 MLP), and 0–5 are Ambiguous: builtin leakage is mitigated but not eliminated by sparse decomposition.

Taken together, the dendrogram view refines the picture from [subsubsection 4.1.3](#). Where geometric structure is sharp — the two tightest dendrogram pairs — the SAE recovers it consistently across depth. Where the dendrogram groups broad families of functionally related but geometrically distinct constructs, SAEs collapse them into early-layer polysemantic bundles that only partially resolve by layer 7. Superposition is not uniform: it is dominant precisely where the model’s own activation geometry is loose, and attenuated where the geometry is tight.

5 Conclusions

Variance partitioning is a lightweight, model agnostic screening tool that requires no architectural changes, no expensive interventions, and no labelled circuits. When applied to a sparse code transformer, it shows that sparsity does more than compress representations — it creates pockets of genuine disentanglement that are invisible in dense models.

The result is a small set of specialised units: MLP neuron #2218 (layer 5) fires 2.84σ above the mean for `Assert` and registers exactly zero across all 31 other classes, while attention head L4H110 is tuned exclusively to import statements, as its nearest rival is an order of magnitude weaker ($\Delta = 12.03$). At

the representational level, the residual stream organises around conceptual rather than syntactic boundaries - `{GeneratorExp, ListComp, SetComp}` cluster by shared iteration semantics, and `{DictComp, Lambda}` reflect a functional-transformation concept learned independently of surface syntax.

These are, however, exceptions. 218 of 271 owned heads are polysemantic, and no residual dimension is exclusive to a single class (Elhage et al., 2022).

SAE analysis independently confirms and extends this picture. 374 of 464 (80.6%) SAE winners replicate externally, and effect sizes grow with depth, peaking mid-late before settling at layer 7. Crucially, SAE winners reflect the model’s own geometry: structurally close pairs like `FunctionDef ↔ AsyncFunctionDef` (cos. dist. 0.04) share a winner at every layer, while distant pairs like `If ↔ For` rarely do.

Polysemantic bundling follows a clear depth profile: broad clusters at early layers - some collapsing 13 constructs onto a single residual feature - progressively narrow to bundles of 2–3 by layer 7, though some clusters remain persistently fused throughout. Factorial controls show that builtin contamination persists even at depth, confirming that sparse decomposition reduces but does not eliminate the AST/builtin entanglement variance partitioning also captures. Where the model’s internal geometry is sharp, sparsity produces genuine disentanglement; where it is loose, superposition remains the dominant encoding strategy.

6 Discussions & Future Work

6.1 Discussions

Dataset design Constructing a stimulus set that fully eliminates confounds proved difficult. Three tensions are hard to resolve simultaneously: many AST constructs require a built-in to be executable (which makes some AST–builtin cross-talk unavoidable); last-token extraction mixes syntactic representation with next-token prediction; and constructs with similar surface structure may drive co-activation regardless of their AST label. Partial remedies include extracting activations at an interior keyword token, applying causal token ablations (Conmy et al., 2023), generating builtin-free prompts where feasible, and diversifying prompt templates.

Applicability to dense models VP’s clean results here depend on two properties dense models lack. AbsTopK gating reduces superposition, so variance attributed to one unit does not leak into correlated neighbours. In a dense model, polysemantic neurons sustain each other under permutation, systematically underestimating the true signal. The natural workaround is to apply VP to SAE latents rather than raw neurons, restoring the near-orthogonality that makes variance attribution interpretable.

Compositionality and interaction neurons A complementary analysis using intersection-based circuit extraction (Gao et al., 2025) reveals a deeper challenge: the Entanglement Index (E_I) averages 0.57 across 1,276 (AST, builtin) pairs, meaning 57% of pair-specific neurons encode the interaction between a construct and its operand type rather than either concept individually. Even a clean AST-unique variance score may therefore contain interaction neurons the builtin regressor does not capture - a third confound that is infeasible to eliminate without a full factorial (AST $\times$ builtin) stimulus design.

6.2 Future Work

The VP provides a lightweight screening pass for mechanistic interpretability, identifying causally relevant units without architectural modifications, and can precede deeper circuit analysis (Conmy et al., 2023) or SAE feature discovery (Cunningham et al., 2023; Templeton et al., 2024). The co-activation findings also point to practical monitoring applications: group-level residual subspaces provide a more robust signal than single neurons, and extending this to safety-relevant constructs e.g. `Import`, `Raise`, exception-handling patterns is a natural next step.

Combining VP screening with the intersection-based marginalisation of Gao et al. (2025) would allow units to be identified cheaply and then verified as encoding syntactic structure rather than AST/builtin terms. The layer-wise bundling profile from the SAE analysis offers a further testable prediction: since polysemantic bundles are broad early and narrow with depth, group-selective dimensions should be most numerous in early layers and most abstract near the output which is something systematic layer-wise VP could directly test.

References

- Yonatan Belinkov. 2022. [Probing classifiers: Promises, shortcomings, and advances](#). *Computational Linguistics*, 48(1):207–219.
- Adly Bricken, Adni Templeton, Joshua Batson, et al. 2024. Scaling and evaluating sparse autoencoders. In *Technical Report, OpenAI*. Describes the use of SAEs to find interpretable features like the "Assert" unit.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebguss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Arthur Conmy, Augustine N. Mavor-Parker, Aengus Lynch, Stefan Heimersheim, and Adrià Garriga-Alonso. 2023. Towards automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems*, volume 36.
- Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. 2023. Sparse autoencoders find highly interpretable features in language models. *arXiv preprint arXiv:2309.08600*.
- Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, Roger Grosse, Sam McCandlish, Jared Kaplan, Dario Amodei, Martin Wattenberg, and Christopher Olah. 2022. [Toy models of superposition](#). *Transformer Circuits Thread*.
- Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Christopher Olah. 2021. [A mathematical framework for transformer circuits](#). *Transformer Circuits Thread*.
- Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. [CodeBERT: A pre-trained model for programming and natural languages](#). In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1536–1547. Association for Computational Linguistics.
- Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. 2024. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*.
- Leo Gao, Achyuta Rajaram, Jacob Coxon, Soham V. Govande, Bowen Baker, and Daniel P. Mossing. 2025. [Weight-sparse transformers have interpretable circuits](#). *ArXiv*, abs/2511.13653.
- Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Wang, and Ming Zhou. 2021. [GraphCodeBERT: Pre-training code representations with data flow](#). In *International Conference on Learning Representations*.
- Stefan Heimersheim and Neel Nanda. 2024. How to use and interpret activation patching. *arXiv preprint arXiv:2404.15255*.
- Nikolaus Kriegeskorte, Marieke M. R. Mur, and Peter A. Bandettini. 2008. [Representational](#)

similarity analysis — connecting the branches of systems neuroscience. *Frontiers in Systems Neuroscience*, 2:4.

Samuel Marks, Can Rager, Eric Todd, Aaron Mueller, Yonatan Belinkov, and David Bau. 2024. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. *arXiv preprint arXiv:2403.19647*.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35, pages 17359–17372.

Kim Nimon, Frederick L. Oswald, and John K. Roberts. 2008. [Regression commonality analysis: A technique for quantitative theory building](#). *Behavior Research Methods*, 40(2):457–466.

Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Christopher Olah. 2022. [In-context learning and induction heads](#). *Transformer Circuits Thread*.

Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Tom Lieberum, Vikrant Varma, János Kramár, Rohin Shah, and Neel Nanda. 2024. Improving dictionary learning with gated sparse autoencoders. *arXiv preprint arXiv:2404.16014*.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian

Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L. Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermy, Shan Carter, Chris Olah, and Tom Henighan. 2024. [Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet](#). *Transformer Circuits Thread*.

Ian Tenney, Dipanjan Das, and Ellie Pavlick. 2019. [BERT rediscovers the classical NLP pipeline](#). In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4593–4601. Association for Computational Linguistics.

A Datasets

A.1 Constructor-Stubs Dataset

The constructor-stubs dataset (introduced in [subsection 3.1.1](#)) is the primary stimulus set for all variance-partitioning, MLP neuron, and attention-head analyses. Each row contains:

- `ast_node`: the target AST node class (e.g., `For`, `If`, `FunctionDef`),
- `builtin_obj`: the paired built-in, or `null` where none is required,
- `prompt_text`: an executable Python snippet containing the target construct,
- `variant_type`: one of `explicit`, `baseline_ast`, `baseline_builtin`, or `proxy`.

The dataset covers 32 AST node classes, spanning statements (`For`, `While`, `If`, `Try`, `With`, `Break`, `Continue`, `Return`, `Raise`, `Delete`, `Global`, `Import`, `Assert`, `AugAssign`, `AnnAssign`), definitions (`FunctionDef`, `AsyncFunctionDef`, `ClassDef`, `Lambda`), expressions (`BoolOp`, `Compare`, `IfExp`, `Subscript`, `GeneratorExp`), comprehensions (`ListComp`, `DictComp`, `SetComp`), yield forms (`Yield`, `YieldFrom`), and synthetic control nodes (`__assign__`, `__raise__`, `__with__`).

Bias-avoidance rationale. Three confounds are addressed in the dataset design.

Syntactic bias. Without lexical variation, activations for a prompt such as `for item in range(10)` conflate the response to the `For` construct with incidental responses to the specific

token item. Six variants per (AST node, builtin) pair (Table 2) average out identifier-level noise, isolating responses invariant to surface form.

Token-prediction bias. Activations are extracted at the *final token*, which risks capturing next-token prediction circuits rather than AST-level representations. DictComp/SetComp prompts would otherwise always end with `}`, and ListComp/Subscript with `]`, so any observed grouping between them could reflect shared delimiter identity. All four are rewritten to constructor form—`list(...)`, `dict(...)`, `set(...)`, `B.__getitem__(0)`—so every prompt ends with `)`.

Built-in co-occurrence. Many AST constructs cannot be instantiated without a builtin: iterative forms require an iterable (`range`), comprehensions require an iterator, and `Assert` idiomatically co-occurs with predicates such as `isinstance` or `len`. Where a builtin is unavoidable, it is included deliberately and paired across 1,405 (AST node, builtin) combinations, with three control conditions per pair: *baseline-AST* (construct retained, builtin replaced by a plain variable); *baseline-builtin* (builtin in a neutral assignment, no target construct); and *proxy* (same semantics without the builtin token, e.g. `data.__len__()` instead of `len(data)`). Where a construct can be expressed without any builtin, the prompt is written without one and no pairing is generated.

A.2 Lexical Variants

Variant	Data var	Result var	Iter var
0	data	result	item
1	items	output	element
2	values	out	x
3	elements	res	entry
4	nums	rv	val
5	seq	acc	node

Table 2: The six lexical variants used per (AST node, builtin) pair. Every explicit prompt is generated once per row, substituting the corresponding identifier set into the template.

B Variance Partitioning

B.1 Design Matrices and Centering

Let $Y \in \mathbb{R}^{N \times D}$ be the activation matrix for N samples (prompts) and D units (e.g., neurons or

attention heads). We first center Y such that each column has a zero mean:

$$Y_c = Y - \bar{Y} \quad (5)$$

Our dataset is described by two categorical factors: the AST node and the Builtin object. Suppose there are k_{ast} unique AST classes and k_{b} unique Builtin classes.

To model the variance explained by each factor, we construct one-hot encoded design matrices. To ensure these matrices are full-rank so that the Ordinary Least Squares (OLS) solution is unique, we drop the first category of each factor to avoid perfect multicollinearity (the dummy variable trap). This dropped category acts as the implicit baseline or reference level.

For a given prompt $i \in \{1, \dots, N\}$ with AST class $A^{(i)} \in \{1, \dots, k_{\text{ast}}\}$ and Builtin class $B^{(i)} \in \{1, \dots, k_{\text{b}}\}$, the elements of our design matrices $X_{\text{ast}} \in \mathbb{R}^{N \times (k_{\text{ast}}-1)}$ and $X_{\text{b}} \in \mathbb{R}^{N \times (k_{\text{b}}-1)}$ are defined as:

$$(X_{\text{ast}})_{i,j} = \mathbb{I}(A^{(i)} = j + 1) \\ (X_{\text{b}})_{i,m} = \mathbb{I}(B^{(i)} = m + 1)$$

where $j \in \{1, \dots, k_{\text{ast}} - 1\}$ and $m \in \{1, \dots, k_{\text{b}} - 1\}$. $\mathbb{I}(\cdot)$ is the indicator function that evaluates to 1 if the prompt belongs to that specific class, and 0 otherwise.

We also define the joint design matrix by horizontally concatenating the two matrices, which captures the combined additive linear effect of both the AST and Builtin factors:

$$X_{\text{joint}} = [X_{\text{ast}} \ X_{\text{b}}] \in \mathbb{R}^{N \times (k_{\text{ast}} + k_{\text{b}} - 2)} \quad (6)$$

B.2 Ordinary Least Squares (OLS) Projection and R^2

For any design matrix X , the OLS projection (hat matrix) H_X computes the fitted values $\hat{Y}_X$:

$$\hat{Y}_X = X(X^T X)^{-1} X^T Y_c \quad (7)$$

For each unit $d \in \{1 \dots D\}$, the total sum of squares (SS_{tot}) and residual sum of squares (SS_{res}) are:

$$SS_{\text{tot}}^{(d)} = \sum_{i=1}^N \left(Y_c^{(i,d)} \right)^2 \\ SS_{\text{res},X}^{(d)} = \sum_{i=1}^N \left(Y_c^{(i,d)} - \hat{Y}_X^{(i,d)} \right)^2$$

The fraction of variance explained by the design matrix X for unit d is:

$$R_X^2(d) = 1 - \frac{SS_{\text{res},X}^{(d)}}{SS_{\text{tot}}^{(d)}} \quad (8)$$

We evaluate this for our three design matrices to obtain R_{ast}^2 , R_{b}^2 , and R_{joint}^2 .

B.3 3. Variance Partitioning

Finally, we decompose the total explained variance into unique, shared, and unexplained components. We apply a clipping function $\max(0, \cdot)$ to bound the values within $[0, 1]$. Write unique-AST as v^{ast} and unique-Builtins as v^{b} , we have:

$$v^{\text{ast}} = \max(0, R_{\text{joint}}^2 - R_{\text{b}}^2) \quad (9)$$

$$v^{\text{b}} = \max(0, R_{\text{joint}}^2 - R_{\text{ast}}^2) \quad (10)$$

$$\text{Shared} = \max(0, R_{\text{ast}}^2 + R_{\text{b}}^2 - R_{\text{joint}}^2) \quad (11)$$

$$\text{Unexplained} = \max(0, 1 - R_{\text{joint}}^2) \quad (12)$$

B.4 Purity Classification Masks

To isolate circuits that strictly process syntactic or semantic information, we categorize each unit d into discrete buckets based on its variance profile. We define a minimum explained variance threshold $\tau = 0.05$ to filter out uninformative units, and a purity ratio $\rho = 0.70$ to ensure a unit is predominantly driven by a single factor.

For each unit d , we assign a boolean classification mask according to the following criteria:

$$\text{AST-Pure} = (v^{\text{ast}} > \tau) \wedge \left(\frac{v^{\text{ast}}}{\max(\epsilon, R_{\text{joint}}^2)} > \rho \right) \quad (13)$$

$$\text{Builtin-Pure} = (v^{\text{b}} > \tau) \wedge \left(\frac{v^{\text{b}}}{\max(\epsilon, R_{\text{joint}}^2)} > \rho \right) \quad (14)$$

$$\text{Shared} = (\text{Shared} > \tau) \wedge (\neg \text{AST-Pure} \wedge \neg \text{Builtin-Pure}) \quad (15)$$

$$\text{Interaction} = (\text{Shared} > \tau) \wedge (R_{\text{joint}}^2 > 2\tau) \quad (16)$$

$$\text{Noise} = (R_{\text{joint}}^2 < \tau) \quad (17)$$

where ϵ is a small constant (e.g., 10^{-9}) added to the denominator to prevent division by zero.

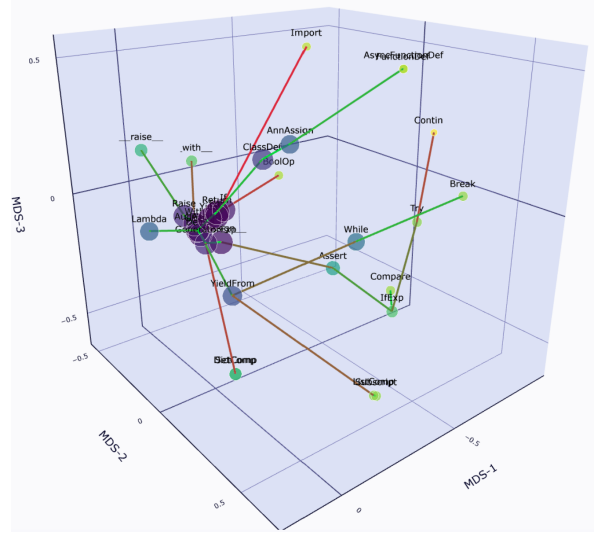


Figure 5: Residual Stream Similarity Graphs using Variance Partitioning Classifications

C SAE

C.1 SAE Architecture

We employ a Top- K Sparse Autoencoder (TopKSAE). Given an input activation $\mathbf{x} \in \mathbb{R}^d$ ($d = 2048$), the SAE computes:

$$\mathbf{x}_c = \mathbf{x} - \mathbf{b}_{\text{dec}}, \quad (18)$$

$$\mathbf{z} = \text{TopK}(\text{ReLU}(\mathbf{W}_{\text{enc}} \mathbf{x}_c + \mathbf{b}_{\text{enc}})), \quad (19)$$

$$\hat{\mathbf{x}} = \mathbf{W}_{\text{dec}} \mathbf{z} + \mathbf{b}_{\text{dec}}, \quad (20)$$

where $\mathbf{W}_{\text{enc}} \in \mathbb{R}^{n_h \times d}$, $\mathbf{W}_{\text{dec}} \in \mathbb{R}^{d \times n_h}$ (column-normalised), and TopK retains only the K largest latent activations.

C.2 Training Details

Separate SAEs are trained for each (layer, site) pair. Activations are extracted from two sites per layer: the **MLP output** (before the residual addition) and the **residual stream** (after attention + MLP + residual connection). Both are harvested simultaneously via forward hooks in one model forward pass per prompt, caching 16 activation tensors plus metadata parquets and the discovery/held-out split manifest.

Each SAE minimises reconstruction MSE with Adam and a cosine LR schedule. Decoder columns are re-normalised to unit norm after each step. The best- R^2 checkpoint on a held-out subset is retained.

C.3 Factorial Feature-Discovery Protocol

Discovery and held-out splits. The 14,683-prompt constructor_stubs

Table 3: SAE hyperparameters.

Parameter	Value
Expansion factor	$8\times$
Hidden dimension n_h	16 384
Top- K sparsity	32
Training epochs	50
Learning rate	10^{-4}
Batch size	256

dataset is partitioned 50/50, stratified by `ast_node $\times$ variant_type`, into a *discovery* split used only for candidate selection and a *held-out* split used for all statistical tests. This partition eliminates winner bias: no feature is reported on the same activations used to select it.

Winner selection. For each (layer, site, AST node) triple, the top-10 candidate features are ranked on the discovery split by $s_j = \bar{z}_j^{(\mathcal{T})} - \bar{z}_j^{(\mathcal{O})}$, the mean-activation gap between target-explicit prompts $\mathcal{T}$ and all other explicit prompts $\mathcal{O}$. The top-1 is the *winner*.

Held-out validation. Each winner is evaluated on the held-out split against five prompt pools: target-explicit, other-explicit, `baseline_ast` (construct retained, builtin replaced), `baseline_builtin` (builtin retained, construct absent), and `proxy` (same semantics, builtin token replaced). Four tests are reported: Mann–Whitney U (one-sided, target-vs-other), Cohen’s d with 95% CI via 1,000-bootstrap, p -value against `baseline_builtin`, and proxy activation retention.

Factorial classification. Each winner receives two orthogonal labels:

- *AST axis* from the baseline contrast: AST-specific (fires on `baseline_ast`, silent on `baseline_builtin`), Builtin-specific (inverse), Both-respond (both active), Joint-required (both silent, target-explicit alone fires), or Ambiguous.
- *Surface axis* from proxy rewrites: Concept (retains activation after builtin token substitution) or Surface-token (activation disappears).

External replication. Winners are re-evaluated on the 8,897-prompt `contrastive_stubs` dataset (explicit variants only, since this dataset

has no baseline/proxy variants). A winner *replicates* if the target-vs-other contrast passes $p < 0.05$ with matching direction.

Null distribution. For every (layer, site, node) triple, a null of 500 random SAE features is constructed and the winner’s specificity ratio percentile reported.

C.4 Dendrogram Corroboration Protocol

A reference dendrogram is computed from the pairwise cosine distances between the mean per-construct post-block residual activations at L7. Ward-linkage clustering yields four coarse clusters used in Section 4.3: *Cyan* (While, Break, Continue, Try, If, For, Import); *Red* (Compare, IfExp, Assert, BoolOp); *Yellow* (FunctionDef, AsyncFunctionDef, AnnAssign, ClassDef, Return); *Purple* (Delete, AugAssign, Global, With, Raise, ListComp, GeneratorExp, SetComp, Subscript, Lambda, DictComp, YieldFrom, Yield). Six *tight pairs* at dendrogram distance ≤ 0.5 are `FunctionDef $\leftrightarrow$ AsyncFunctionDef` (0.04), `ListComp $\leftrightarrow$ GeneratorExp` (0.07), `AnnAssign $\leftrightarrow$ ClassDef` (0.14), `While $\leftrightarrow$ Break` (0.20), `Compare $\leftrightarrow$ IfExp` (0.36), `If $\leftrightarrow$ For` (0.48).

For each (layer, site), the SAE top-1 winner for every construct is collected and inverted: *feature* $\rightarrow$ *nodes sharing that winner*. A tight pair is *corroborated* iff the two constructs share the same winner; a cluster is *bundled* iff at least two of its members do. The *bundle size* is the largest such within-cluster intersection.

D Unique MLP Neurons

D.1 MLP Neuron Identification

MLP neuron identification follows three stages. **Stage 1 — VP purity.** For each neuron u at layer l , OLS hat-matrix projection yields unique-variance components $\Delta_{\text{ast}} = R_{\text{joint}}^2 - R_{\text{builtin}}^2$ and $\Delta_{\text{builtin}} = R_{\text{joint}}^2 - R_{\text{ast}}^2$. A neuron is AST-pure if $\Delta_{\text{ast}}(u, l) > 0.10$ and $\Delta_{\text{ast}}(u, l)/R_{\text{joint}}^2(u, l) > 0.70$, ensuring its variance is attributable to AST class identity rather than builtin co-variation. **Stage 2 — Selectivity.** For each AST-pure neuron we compute

$$\text{sel}(u, c, l) = \frac{\bar{a}_{u,l}|_c - \bar{a}_{u,l}}{\sigma_{u,l} + \varepsilon}, \quad (21)$$

where $\bar{a}_{u,l}|_c$ is the mean activation over all prompts of class c , and $\bar{a}_{u,l}$, $\sigma_{u,l}$ are the global

mean and standard deviation across all classes. **Stage 3 — Monosemanticity.** A neuron is strictly monosemantic if $\text{sel}_{\text{top1}} > 0.1$ and $\text{sel}_{\text{top2}} < 0.01$, i.e. it fires meaningfully for exactly one class with near-zero activation on all others. This hard silencing is a structural consequence of the model’s AbsTopK gating, which zeroes any neuron outside the top- k pre-activation magnitudes, suppressing cross-class leakage characteristic of superposition.

D.2 Monosemantic MLP Neurons

Applying the criterion above, only three neurons satisfy strict monosemanticity across all 32 AST classes (Table 4).

Layer	Neuron	Owner class	sel
L5	N2218	Assert	2.839
L6	N1409	With	1.247
L6	N6013	__assign__	0.190

Table 4: Strictly monosemantic MLP neurons. Each neuron fires for exactly one AST class with zero activation on all 31 remaining classes. Runner-up class for all three neurons is `__with__`, with < 0.01 selectivity, confirming the absence of any meaningful response to non-owner classes.

E Unique Attention Heads by Class

The same variance-partitioning purity criterion described in Section D.1 is applied to attention heads: a head (l, h) is deemed AST-pure if

$$v_{l,h}^{\text{ast}} > 0.10 \quad \text{and} \quad \frac{v_{l,h}^{\text{ast}}}{R_{\text{joint},l,h}^2} > 0.70, \quad (22)$$

i.e. at least 70% of its explained variance arises from AST identity alone. Applied to the constructor-stubs dataset this retains 316 AST-pure heads out of 1024 total. Per-class selectivity $\text{sel}(h, c, l)$ is computed identically to Section D.1; a head is assigned ownership of every class c for which $\text{sel}(h, c, l) > 0.4$.

i

E.1 Monosemantic Attention Heads

Of the 316 AST-pure heads, 271 pass the $\text{sel} > 0.4$ ownership threshold for at least one class; 218 are multi-class (owned by two or more classes simultaneously) and 53 are strictly monosemantic (single owner). The remaining 45 AST-pure heads

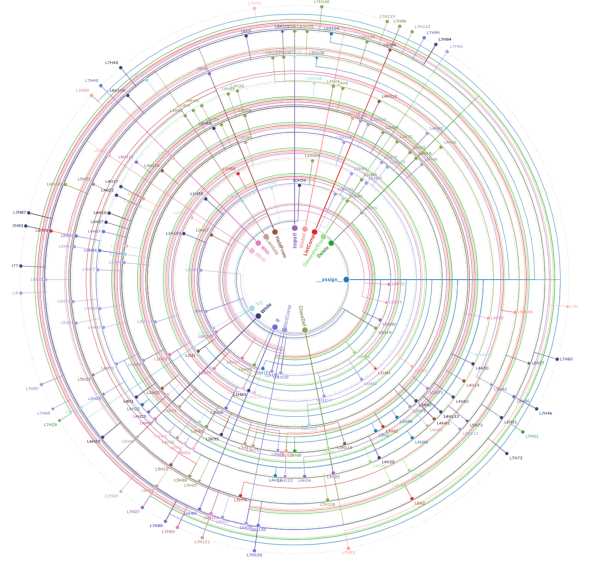


Figure 6: Interactive tool for analysing attention head co-activation and uniqueness

are unowned at this threshold. Table 5 lists the 53 monosemantic heads ranked by selectivity gap $\Delta = \text{sel}_{\text{top1}} - \text{sel}_{\text{top2}}$.

Table 5: Monosemantic attention heads (53 single-owner out of 271 owned, 316 AST-pure total; 218 multi-class). Ranked by selectivity gap $\Delta = \text{sel}_{\text{top1}} - \text{sel}_{\text{top2}}$.

Layer	Head	Owner class	sel	Runner-up class	Runner-up sel	Gap	Verdict
L4	H110	Import	12.138	Delete	0.110	12.029	Genuinely exclusive
L5	H107	SetComp	4.878	AugAssign	0.058	4.820	Genuinely exclusive
L0	H16	Subscript	4.301	Global	0.330	3.971	Genuinely exclusive
L6	H102	While	3.813	__with__	0.000	3.813	Genuinely exclusive
L6	H25	DictComp	3.743	__with__	0.000	3.743	Genuinely exclusive
L4	H106	Try	3.608	__with__	0.000	3.608	Genuinely exclusive
L0	H7	Try	3.604	__with__	0.000	3.604	Genuinely exclusive
L0	H79	DictComp	3.751	Raise	0.199	3.552	Genuinely exclusive
L6	H29	Try	3.607	If	0.361	3.246	Genuinely exclusive
L6	H18	DictComp	2.963	ClassDef	0.084	2.879	Genuinely exclusive
L0	H42	DictComp	3.152	Raise	0.378	2.773	Genuinely exclusive
L5	H52	YieldFrom	2.749	__with__	0.000	2.749	Genuinely exclusive
L0	H122	SetComp	2.844	Raise	0.184	2.659	Genuinely exclusive
L4	H103	DictComp	2.931	Raise	0.283	2.648	Genuinely exclusive
L2	H76	DictComp	2.824	Raise	0.188	2.637	Genuinely exclusive
L5	H89	DictComp	2.907	Raise	0.339	2.568	Genuinely exclusive
L6	H66	DictComp	2.650	GeneratorExp	0.097	2.552	Genuinely exclusive
L0	H0	Subscript	2.661	Yield	0.318	2.343	Genuinely exclusive
L5	H13	Try	2.474	If	0.383	2.091	Genuinely exclusive
L4	H91	Try	2.272	Yield	0.304	1.967	Genuinely exclusive
L5	H43	Lambda	2.043	__raise__	0.094	1.950	Genuinely exclusive
L3	H55	GeneratorExp	1.984	Lambda	0.120	1.864	Genuinely exclusive
L5	H3	Subscript	2.102	AnnAssign	0.311	1.791	Genuinely exclusive
L7	H21	Global	1.960	For	0.255	1.705	Genuinely exclusive
L0	H15	DictComp	1.780	If	0.201	1.579	Genuinely exclusive
L6	H82	DictComp	1.769	Assert	0.207	1.562	Genuinely exclusive
L3	H52	IfExp	1.780	Continue	0.314	1.466	Genuinely exclusive
L6	H64	DictComp	1.693	Lambda	0.237	1.456	Genuinely exclusive
L5	H29	If	1.793	While	0.353	1.439	Genuinely exclusive
L7	H69	Lambda	1.709	Global	0.383	1.326	Genuinely exclusive
L5	H76	AugAssign	1.507	Global	0.350	1.157	Genuinely exclusive
L1	H70	Delete	1.394	Subscript	0.394	1.001	Genuinely exclusive
L5	H106	__assign__	1.031	For	0.167	0.863	Threshold-dependent
L0	H6	SetComp	1.131	AnnAssign	0.367	0.763	Threshold-dependent
L4	H36	AugAssign	1.048	YieldFrom	0.347	0.701	Threshold-dependent
L4	H86	AugAssign	0.978	ListComp	0.370	0.607	Threshold-dependent
L7	H85	Import	0.754	AnnAssign	0.252	0.502	Threshold-dependent
L7	H29	Delete	0.607	Global	0.161	0.446	Borderline
L5	H18	ListComp	0.737	GeneratorExp	0.308	0.429	Borderline
L0	H14	With	0.699	Try	0.374	0.324	Borderline
L6	H28	Raise	0.631	While	0.331	0.300	Borderline
L4	H95	Lambda	0.653	__raise__	0.360	0.293	Borderline
L2	H101	GeneratorExp	0.560	Lambda	0.277	0.283	Borderline
L4	H121	Raise	0.597	YieldFrom	0.333	0.264	Borderline
L5	H100	__raise__	0.485	Import	0.231	0.254	Borderline
L5	H73	While	0.440	Import	0.233	0.207	Borderline
L7	H98	BoolOp	0.479	Lambda	0.319	0.159	Borderline
L5	H51	AugAssign	0.407	Raise	0.252	0.155	Borderline
L6	H4	Lambda	0.538	ClassDef	0.399	0.139	Borderline
L2	H83	Global	0.505	ClassDef	0.400	0.105	Borderline
L6	H67	While	0.422	For	0.340	0.082	Borderline
L6	H79	AnnAssign	0.434	With	0.377	0.058	Borderline
L4	H29	Raise	0.417	Return	0.371	0.046	Borderline

F Unique Residual Stream Neurons by Groups

Ward-linkage hierarchical clustering is applied to the mean per-class residual activations at $k=14$. For each resulting group, a group-selectivity score $\Delta\bar{s} = \bar{s}_G - \bar{s}_{\bar{G}}$ and within-group variance σ_G are computed; significance is assessed by permutation test (5,000 relabellings, $p < 0.01$). Residual dimensions satisfying $\Delta\bar{s} > 0.5$, $\sigma_G < 0.8$, and $p < 0.01$ are counted as NumSig. Ten of fourteen groups yield significant selective units; three singletons (BoolOp, Import, __with__) produce none, and Section 4.1.3 discusses the two groups of particular interpretive interest.

Group	AST classes	NumSig	Top unit	$\Delta\bar{s}$	σ_G	p
G1	Yield, YieldFrom	244	L7N1732	3.703	0.055	<0.001
G2	GeneratorExp, ListComp, SetComp	1097	L4N1327	2.980	0.356	<0.001
G3	Subscript, __assign__	40	L3N1489	1.557	0.098	0.002
G4	Raise, __raise__	577	L5N1123	2.751	0.708	0.002
G5	DictComp, Lambda	563	L1N1194	2.500	0.373	0.002
G6	AugAssign, Delete, Global, With	310	L8N1902	1.374	0.081	<0.001
G7	AnnAssign, AsyncFunctionDef, ClassDef, FunctionDef, Return	2009	L4N1210	2.178	0.270	<0.001
G8	Assert, Compare, IfExp	1091	L6N1795	1.992	0.069	<0.001
G9	BoolOp	0	—	—	—	—
G10	Break, While	811	L4N1633	3.602	0.738	0.003
G11	Continue, Try	695	L7N528	2.657	0.320	0.002
G12	For, If	17	L4N1878	2.117	0.776	0.004
G13	Import	0	—	—	—	—
G14	__with__	0	—	—	—	—

Table 6: Group-selective residual stream dimensions at $k=14$ ($\Delta\bar{s} > 0.5$, $\sigma_G < 0.8$, $p < 0.01$, core excluded). Groups with NumSig = 0 produced no units at this threshold.